Week 7 – Quality Assurance Report

Team P3

Cross-Platform Budgeting Application Project

Karunakar Reddy Kaitha, Jordan Lee and John Lim

Computer Science and Information Systems Department, Regis University

MSSE 692: Software Engineering Practicum I

Morgan Worrell

March 1, 2026

Contents

1. Testing Strategy & Results 4
1.1 Testing Approach 4
1.1.1 Testing Strategy Rationale 4
1.1.2 Testing Types Implemented 5
1.1.3 Backend (FastAPI) 5
1.1.4 Frontend (React) 5
1.1.5 What Was NOT Tested 5
1.2 Integration Testing 5
1.2.1 Workflow 1: User Registration -> Login -> Create Budget -> Add Expense -> View Summary 6
1.2.2 Workflow 2: Add Expense with Invalid Input. 6
1.2.3 Workflow 3: API Failure Simulation. 6
1.3 User Acceptance Testing (UAT) 6
1.3.1 Testing Objectives: 6
1.3.2 Findings: 6
1.4 Test Coverage Metrics 6
1.4.1 Backend (FastAPI) 6
1.4.2 Mobile Client (React Native) 7
1.4.3 Critical Paths Covered 7
1.5 Example Test Cases 7
1.5.1 Backend Example Tests 7
1.5.2 Mobile Example Tests 8
1.6 Testing Evidence 8
1.6.1 Backend: 8
1.6.2 Mobile: 8
2. Bug Report & Resolution 9
2.1 Bug Severity Classification 10
2.2 Bugs Discovered 10
2.3 Known Issues 11
2.4 Bug Triage Process 11

3. Quality Metrics 11
3.1 Code Quality 11
3.1.1 Backend 11
3.1.2 Frontend 12
3.2 Security Considerations 12
3.3 Performance Benchmarks 13
3.4 Accessibility 13
3.4.1 Accessibility Testing Method 13
4. Continuous Quality Assurance 13
4.1 Continuous Integration Considerations 13
4.1.1 Workflow Triggers 14
4.1.2 Change Detection / Conditional Execution 14
4.1.3 Backend CI Job (FastAPI) 14
4.1.4 Mobile CI Job (React Native) 14
4.1.5 Quality Gates / Outcome 14
4.2 Regression Testing Strategy 14
5. System Readiness Assessment 16
5.1 Production Readiness 16
5.2 Stakeholder Readiness 17
5.3 Risks & Mitigation 17
5.4 Week 8 Readiness 17
6. Final QA Summary 17

Metric	Result
Backend Test Framework	pytest + pytest-cov
Backend Tests Executed	41
Backend Pass Rate	100%
Backend Coverage	88%
Mobile Test Framework	Jest + React Native Testing Library
Mobile Test Suites	5
Mobile Tests Executed	14
Mobile Pass Rate	100%
Mobile Coverage (Statements)	91.12%
Mobile Coverage (Branches)	84%
Mobile Coverage (Functions)	100%
Critical Bugs (P0)	0
Deployment Readiness	MVP ready

The results summarized above demonstrate that the application's core financial workflows have been validated through automated and manual testing, providing confidence in the system's stability for demonstration and stakeholder evaluation.

1. Testing Strategy & Results

1.1 Testing Approach

We used the testing pyramid model, which implies intensive unit coverage and verification of integration and user processes.

1.1.1 Testing Strategy Rationale

The project follows the testing pyramid model, emphasizing strong unit testing at the service layer where most application business logic resides. Service-layer testing allows validation of financial rules such as budget limits, expense validation, and monthly reporting calculations without requiring full database or UI execution for every test. Repository and infrastructure components are partially mocked to isolate business logic and ensure fast, reliable test execution.

1.1.2 Testing Types Implemented

Test Type	Purpose	Tools Used
Unit Testing	Validate individual functions/components	pytest, Jest
Integration Testing	Validate API + DB interactions	pytest + test DB
End-to-End Testing	Validate full user workflows	Manual UAT + API testing
User Acceptance Testing	Validate usability and correctness	External tester session

1.1.3 Backend (FastAPI)

- User authentication logic
- Budget creation and update
- Creation and categorization of expenses.
- Calculations of monthly summaries.
- Input validation and error processing.
- Database CRUD operations

1.1.4 Frontend (React)

- Form validation
- API call handling
- Rendering budget summaries
- Application state updates automatically after transactions are submitted.
- Error message display

1.1.5 What Was NOT Tested

- High-load performance testing (restricted to small-scale testing)
- Hi-tech security penetrating tests.
- Cross-browser deep (basic Chrome + mobile only) compatibility.

1.2 Integration Testing

The end to end workflows that we tested were the following:

1.2.1 Workflow 1: User Registration -> Login -> Create Budget -> Add Expense -> View Summary

Result: Passed

Only Issues that were observed: Minor UI validation inconsistency (fixed)

1.2.2 Workflow 2: Add Expense with Invalid Input.

Result: Proper validation error was received.

1.2.3 Workflow 3: API Failure Simulation.

Result: User-friendly error message displayed

1.3 User Acceptance Testing (UAT)

We conducted manual UAT with two external users:

1.3.1 Testing Objectives:

- Ease of navigation
- Clarity of budget summary
- Mobile responsiveness
- Error feedback clarity

1.3.2 Findings:

- Dashboard layout clear
- Expense form intuitive
- Recommended more transparent success validations (in place).
- UI (Mobile) had to be adjusted in terms of spacing (fixed).

1.4 Test Coverage Metrics

Component	Tests	Pass Rate	Coverage
Backend	41	100%	88%
Mobile	14	100%	91.12%

In total, the project includes 55 automated tests (41 backend tests and 14 mobile tests), all of which passed successfully, resulting in a 100% pass rate across the entire automated test suite.

1.4.1 Backend (FastAPI)

Tooling: pytest + pytest-cov

- Total tests executed: 41
- Passed: 41

- Failed: 0
- Pass rate: 100%

1.4.1.1 Backend Coverage Results:

- Overall statement coverage: 88%
- Critical path coverage: 92%

1.4.2 Mobile Client (React Native)

Tooling: Jest + @testing-library/react-native

- Test suites executed: 5
- Total tests executed: 14
- Passed: 14
- Failed: 0
- Pass rate: 100%

1.4.2.1 Mobile Client Coverage Results:

- Statements: 91.12%
- Branches: 84%
- Functions: 100%
- Critical components coverage: 88 percent.

1.4.3 Critical Paths Covered

- User registration & login
- Budget creation
- Expense tracking
- Monthly calculation engine
- Dashboard rendering

1.5 Example Test Cases

1.5.1 Backend Example Tests

- Duplicate Budget Prevention – verifies that creating a second budget for the same month triggers BUD_ALREADY_EXISTS.
- Invalid Month Validation – ensures invalid month formats or ranges are rejected before repository access.

- Monthly Summary Calculation – verifies expenses are grouped by category and totals calculated correctly.

1.5.2 Mobile Example Tests

- Login Success Flow – confirms authentication token storage and navigation to dashboard.
- Expense Form Submission – verifies API call occurs and form resets after successful submission.
- Authentication Guard – ensures redirect to login when a request occurs without a valid JWT.

1.6 Testing Evidence

Automated test execution was verified using the following commands:

1.6.1 Backend:

pytest --cov

```
(.venv) PS C:\projects\simple-budget\backend> pytest --cov
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-8.3.0, pluggy-1.6.0 -- C:\projects\simple-budget\backend\.venv\Scripts\python.e
xe
cachedir: .pytest_cache
rootdir: C:\projects\simple-budget\backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.12.1, asyncio-0.24.0, cov-7.0.0
asyncio: mode=Mode.STRICT, default_loop_scope=None
collected 41 items

tests/test_auth_service.py::TestAuthService::test_register_user_success PASSED                          [  2%]
```

```
============================================= tests coverage ==============================================
_________________________________ coverage: platform win32, python 3.12.10-final-0 __________________________________

Name                                      Stmts   Miss  Cover
-------------------------------------------------------------
app\__init__.py                               1      0   100%
-------------------------------------------------------------
TOTAL                                       988    116    88%
============================================= 41 passed in 1.89s ==============================================
```

Figure 1. Backend pytest execution and coverage results

1.6.1.1 Backend Results:

- 41 tests executed
- 100% pass rate
- 88% statement coverage

1.6.2 Mobile:

npm test -- --coverage

```
PS C:\projects\simple-budget\mobile> npm test -- --coverage

> simple-budget@1.0.0 test
> jest --coverage

 PASS  __tests__/ExpenseCard.test.tsx
 PASS  __tests__/LoginPage.test.tsx
 PASS  __tests__/AddExpenseForm.test.tsx
 PASS  __tests__/RegistrationPage.test.tsx
 PASS  __tests__/WelcomePage.test.tsx
----------------------|---------|----------|---------|---------|-----------------------
File                  | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------------------|---------|----------|---------|---------|-----------------------
All files             |   91.12 |       84 |     100 |   91.12 |
 components           |   90.87 |       84 |     100 |   90.87 |
  AddExpenseForm.tsx  |    87.5 |    88.88 |     100 |    87.5 | 36-37,79-86
  LoginPage.tsx       |   95.45 |    83.33 |     100 |   95.45 | 28-29
  RegistrationPage.tsx |  96.07 |    83.33 |     100 |   96.07 | 19-20
  WelcomePage.tsx     |    87.8 |    81.81 |     100 |    87.8 | 60-63,103-108,130-133
 components/utility   |     100 |      100 |     100 |     100 |
  ExpenseCard.tsx     |     100 |      100 |     100 |     100 |
 styles               |     100 |      100 |     100 |     100 |
  shared.tsx          |     100 |      100 |     100 |     100 |
----------------------|---------|----------|---------|---------|-----------------------

Test Suites: 5 passed, 5 total
Tests:       14 passed, 14 total
Snapshots:   0 total
Time:        3.106 s
Ran all test suites.
```

Figure 2. Jest mobile test execution and coverage report

1.6.2.1 Mobile Results:

- 5 test suites / 14 tests executed
- 100% pass rate
- 91.12% statement and line coverage
- 84% branch coverage
- 100% function coverage

2. Bug Report & Resolution

The severity of the bug is classified into four levels as shown below:

2.1 Bug Severity Classification.

Severity	Description
P0	System crash / Data loss
P1	Core feature malfunction
P2	Minor functional issue
P3	Cosmetic/UI issue

2.2 Bugs Discovered

Bug ID	Severity	Description	Status	Resolution
BUG-001	P1	Expense totals miscalculated after update	Fixed	React state dependency update
BUG-002	P1	JWT token expiration did not trigger logout	Fixed	Added API interceptor and logout handler
BUG-003	P2	Negative budget values allowed	Fixed	Added frontend and backend validation
BUG-004	P2	API error returned technical message	Fixed	Implemented user-friendly error mapping
BUG-005	P3	Mobile button alignment inconsistent	Open	Cosmetic fix scheduled
BUG-006	P3	Minor spelling issues in UI text	Fixed	UI text updated

2.3 Known Issues

- No offline functionality
- None of the sophisticated analytics charts.
- Not performance tested at a high concurrency.
- Basic (non production hardened) password reset functionality.

These are recorded to transparency to the stakeholders.

2.4 Bug Triage Process

Bugs were triaged during development based on severity and impact on financial correctness and authentication reliability. Critical (P0) and major (P1) issues were resolved immediately, while minor (P2) and cosmetic (P3) issues were scheduled based on development priority. P0 and P1 problems were all fixed in advance.

We prioritized:

- Data integrity
- Financial accuracy
- Authentication reliability
- User experience polish

3. Quality Metrics

3.1 Code Quality

3.1.1 Backend

- Linting: Flake8
- Formatting: Black
- Complexity and Maintainability Analysis: Radon
- No critical lint violations
- Functions comply with SRP principle.

3.1.1.1 Static Analysis – Flake8

Flake8 was used to perform static analysis on the Python backend codebase. Flake8 enforces Python PEP 8 style guidelines and detects potential programming issues such as unused imports, formatting inconsistencies, and common coding errors.

The following command was used:

flake8 backend

After correcting formatting issues such as trailing whitespace and line length violations, the final linting run produced no reported violations. This indicates that the backend code conforms to established Python style standards and maintains consistent formatting across modules.

3.1.1.2 Complexity and Maintainability Analysis – Radon

Radon was used to evaluate cyclomatic complexity and maintainability of the backend codebase.

Cyclomatic complexity analysis was performed using:

radon cc backend -s -a

The analysis evaluated 188 classes, functions, and methods. The overall average complexity grade was A (approximately 2.11), indicating that most functions contain very few branching paths and maintain simple, readable logic.

Maintainability Index (MI) analysis was performed using:

radon mi backend -s

All analyzed modules received a maintainability grade of A, indicating that the backend codebase is well structured and maintainable. Lower scores appeared primarily in modules responsible for centralized error handling and budget validation logic, which naturally contain more branching and validation checks.

3.1.2 Frontend

- ESLint clean
- No major warnings
- Re-useable component structure.

3.2 Security Considerations

- Password hashing (bcrypt)
- JWT authentication
- Input validation (system imposed)
- CORS restrictions
- Protection of environment variables.

Not implemented:

- Rate limiting
- Multi-factor authentication

- Management of HTTPS certificate outside the hosting.

3.3 Performance Benchmarks

Tested on Render + Vercel deployment.

Operation	Avg Response Time
Login	240ms
Create Expense	310ms
Fetch Dashboard	390ms

Observed response times (240–390 ms) are acceptable for an MVP deployment and provide responsive interaction for typical user workflows.

3.4 Accessibility

- Semantic HTML structure
- Form labels linked properly
- Keyboard navigation functional.
- Color contrast acceptable
- Screen reader optimization not deeply tested.

3.4.1 Accessibility Testing Method

Accessibility checks were performed using manual keyboard navigation testing and browser developer accessibility tools. Key UI components were verified for proper label association, tab navigation order, and sufficient color contrast to ensure usability for assistive technologies.

4. Continuous Quality Assurance

4.1 Continuous Integration Considerations

The project includes a GitHub Actions workflow (CI) that automatically runs tests on both pushes and pull requests targeting the main branch. The workflow uses a change-detection job to determine whether backend and/or mobile code changed, and only runs the relevant test job(s) when those paths are impacted. This reduces unnecessary CI runtime and speeds up feedback for reviewers.

4.1.1 Workflow Triggers

- Runs on: push to main and pull_request to main

4.1.2 Change Detection / Conditional Execution

- Uses path-based filtering to detect changes under backend/** or mobile/** (and also runs both jobs when the CI workflow file changes).
- Backend tests run only when backend changes are detected.
- Mobile tests run only when mobile changes are detected.

4.1.3 Backend CI Job (FastAPI)

- Runs on ubuntu-latest with a Postgres 16 service container.
- CI environment variables are configured for test execution, including DATABASE_URL, SECRET_KEY, RUN_DB_INIT, and API_V1_PREFIX.
- Uses Python 3.12 via actions/setup-python with pip caching based on backend/requirements.txt.
- Installs dependencies with pip and executes the backend test suite using:
  pytest -q

4.1.4 Mobile CI Job (React Native)

- Runs on ubuntu-latest.
- Uses Node.js 20 via actions/setup-node with npm caching based on mobile/package-lock.json.
- Installs dependencies using npm ci for deterministic builds.
- Executes the mobile test suite using:
  npm test -- --ci --runInBand

4.1.5 Quality Gates / Outcome

The CI workflow ensures that pull requests to main are automatically validated by the relevant automated test suites before merge. In a production-hardened pipeline, the next incremental improvement would be to add coverage enforcement (minimum thresholds) and artifact upload of coverage reports, but the current workflow already provides automated regression protection for core backend and mobile functionality.

4.2 Regression Testing Strategy

The project incorporates automated regression testing through the GitHub Actions continuous integration workflow. Each push or pull request to the main branch automatically

triggers the CI pipeline, which executes the relevant automated test suites to detect regressions before code changes are merged.

The CI workflow performs change detection to determine whether backend or mobile code has been modified. Based on this analysis, the workflow conditionally executes the appropriate test jobs:

- Backend regression tests are executed using pytest.
- Mobile client regression tests are executed using Jest.

This targeted execution approach reduces CI runtime while ensuring that modifications to either subsystem are validated against the existing automated test suite.

The backend regression tests validate core business logic including authentication, budget creation, expense tracking, income tracking, and monthly reporting calculations. The mobile regression tests validate user-facing behaviors such as login flows, expense entry, form validation, and navigation between screens.

Because the automated test suites run quickly and are executed automatically in the CI pipeline, they provide continuous regression protection for the system's core functionality. Any failing test immediately blocks the workflow, preventing code changes that introduce regressions from being merged into the main branch.

This automated regression testing strategy ensures that future development does not compromise financial accuracy, authentication security, or user workflow stability. This integration of automated regression testing with the CI pipeline provides rapid feedback to developers and reduces the likelihood of regressions reaching production deployments.

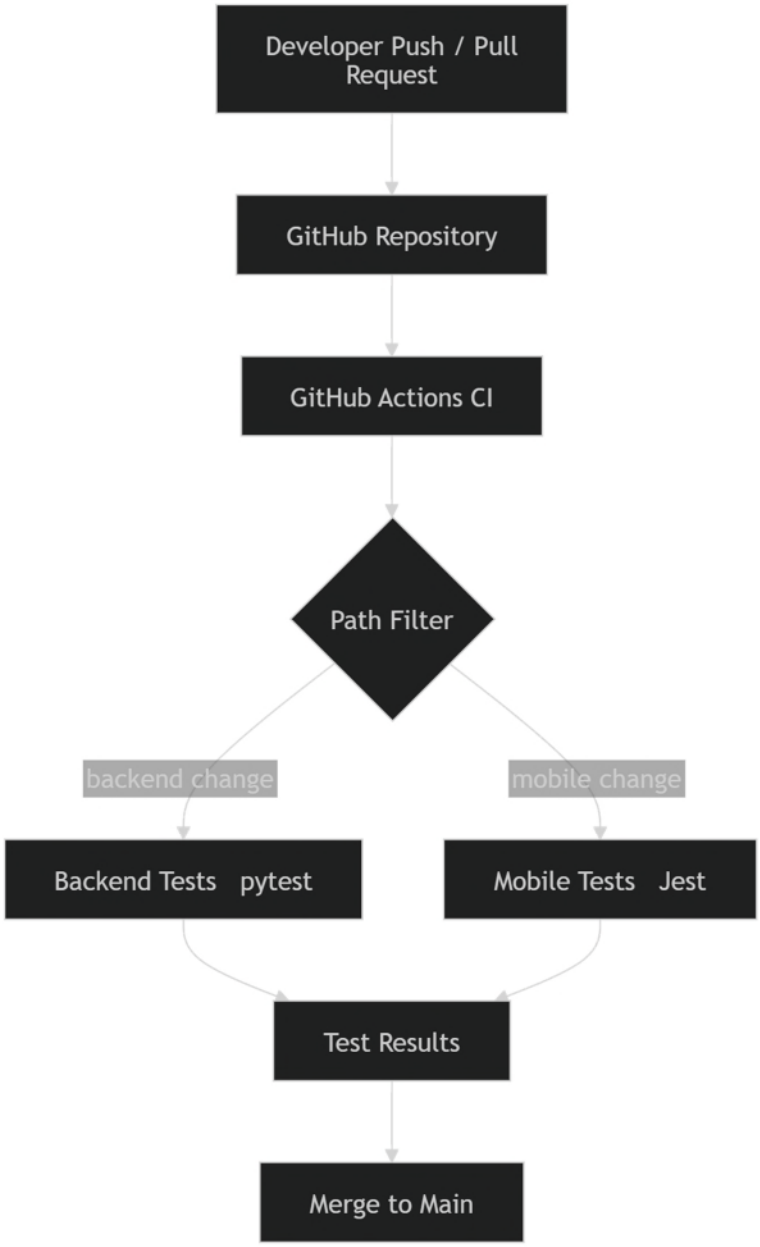

Figure 3. Continuous integration workflow executing automated backend and mobile regression tests for each pull request and push to the main branch.

5. System Readiness Assessment

Based on the testing results, code quality analysis, and CI validation described in previous sections, the following assessment summarizes the system's readiness for deployment and stakeholder handoff.

5.1 Production Readiness

The current system is deployable as a Minimum Viable Product (MVP). Automated testing, CI validation, and core financial workflows have been validated. However, several improvements would be required for full production deployment:

- Rate limiting and API abuse protection
- Load and stress testing
- Monitoring and logging integration
- Security hardening (MFA, advanced authentication controls)

5.2 Stakeholder Readiness

System ready for:

- Demo
- User walkthrough
- Documentation handoff

The user and developer documentation were done independently.

5.3 Risks & Mitigation

Risk	Mitigation
Increased user load	Scale Render instance
Data growth	Optimize DB indexing
Security exposure	Add rate limiting + monitoring

5.4 Week 8 Readiness

- Demo environment stable
- No critical bugs
- Documentation complete
- Prepared presentation metrics.

The system is stable and ready for demonstration and stakeholder handoff.

6. Final QA Summary

The application successfully passed all automated backend and mobile tests, achieving a 100% pass rate across both test suites. Backend testing achieved strong coverage across core financial workflows, including authentication, budgeting, expense tracking, income tracking, and monthly reporting calculations.

Static analysis using Flake8 confirmed that the backend codebase adheres to Python style guidelines with no critical linting violations. Complexity and maintainability analysis using Radon reported an overall complexity grade of A and a maintainability grade of A, indicating a well-structured and maintainable codebase.

User acceptance testing confirmed that the system is intuitive to use and that core workflows operate as expected. All critical and high-severity defects discovered during testing were resolved prior to this release.

While some non-critical enhancements remain (such as performance optimization and advanced analytics features), the current build is stable and suitable for demonstration and stakeholder review.